Cover page

Declaration of Originality

Proforma

Contents

List of Figures

1 Introduction

1.1 The Problem in a Question

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing they have been proven to provide system-wide security properties such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5].
The employment of a tagged memory architecture can certainly provide utility, but there are associated costs that render widespread adoption of tagged memory architectures currently infeasible. With a simplistic scheme, the number of memory accesses that workloads must perform doubles due to the CPU now having to access the tag of the value it writes, as well as the value itself, which brings an unacceptably large time overhead. Also, the table in which tags are stored comes with a constant memory consumption cost, which is particularly disadvantageous within the server-side compute industry, where every byte of DRAM is precious.

Efforts have been and are currently being made to mitigate the overheads of tagged memory. The question that this dissertation is concerned with is:

"How can we further reduce the overheads that tagged memory introduces?"

1.2 Potential Answers

Tag caches are often used in practice to hold tag data close to the CPU for faster access times. The main area in which mitigation of the memory access latency overhead is found is in increasing the 'cacheability' of the tag table in DRAM. That is, by increasing the amount of information about the tags that we store in a fast-access cache, we reduce the average latency overhead that is introduced by tagged memory's new requirement of accessing tag data, as well as value data. An effort of this kind, described in *Efficient Tagged Memory*, has been shown to reduce the common-case DRAM traffic overhead to "well below 1%".
Also, suppose that the tagged memory system is one which runs a hypervisor, which is a common setup for servers. Suppose further that the hypervisor has a dynamic DRAM allocator that could offer free physical address regions to its guest operating systems as and when they require it. A novel system involving a dynamically-resizing tag table that can be compressed and decompressed at runtime would allow the guest OSes to reserve and free regions of DRAM that they use for storing their tags as they execute, meaning that each OS will consume large amounts of extra space only when necessary, as opposed to all of them reserving at boot-time the maximum that they might require during any part of their execution. By increasing the 'compressibility' of the tag table, we can keep the size of the tag table in each guest OS instance closer to a theoretical minimum, given the current workloads each is performing.

1.3 My Project

These potential areas of improvement motivate the exploration and investigation of existing tag controller schemes in search of opportunities to improve upon them, to increase the 'cacheability' and 'compressibility' of the DRAM tag table. They also motivate the creation of tools that allow architects to quantitatively and qualitatively evaluate new tag controller schemes that they might design.
My project aims to explore existing tag controller implementations, as well as provide a framework for an architect to implement a controller of their own design, which provides them with qualitative and quantitative data that allows them to usefully investigate and understand its performance, for the sake of contributing to the field of research in this area, and participating in answering the question I posed earlier.

2 Preparation

This chapter explains the relevant background material required to undertake the project and to understand the dissertation. It also details the major pieces of my project, how they fit together, and what was required to produce a successful implementation.

2.1 Background Material

2.1.1 Tagged Memory Architectures

In a tagged memory architecture N-bit values in memory are notionally 'extended' with M-bit tags. Doing so can provide desirable security properties like those mentioned in the introduction. *Efficient Tagged Memory* [1] describes how a one-bit tag is sufficient to implement all of the security properties that tagged memory architectures can afford. The extension of values with tags might be implemented in one of several ways, but section III of the paper details how storing all of the tag bits in a single table at the top of DRAM is the most practical for single-bit tag (SBT) architectures; this is also the most used technique in practice.
These architectures mandate allocation of a static region of DRAM at boot-time to act as the tag table. This results in a constant physical memory consumption cost that is proportional to the size of the (taggable) address space.

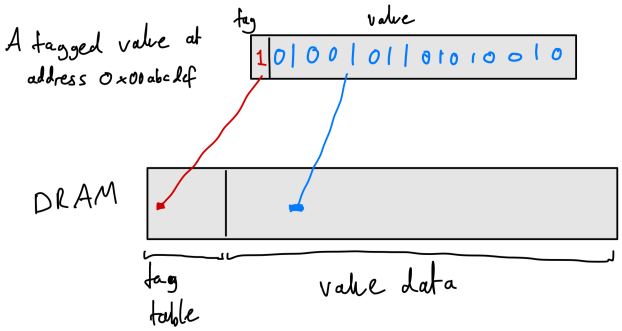

Figure 1: How the two parts of a tagged value are stored in physical memory.

To implement such a scheme in hardware, we use a tag controller. This unit is responsible for intercepting memory access requests and translating them into appropriate requests that will result in the retrieval of both the value and its tag.

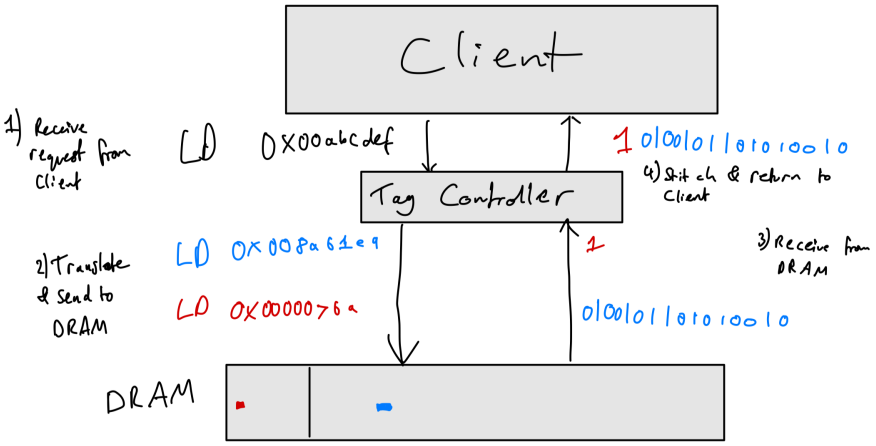

Figure 2: Step-by-step memory access translation.

My project focuses on merged-cache hierarchy architectures. In these, values in the cache are stored unified with their tag – that is, the data caches contain tags as well as values. Section IV of *Efficient Tagged Memory* explains how the alternative, split-cache hierarchy architectures, are 'unnecessary', and result in a less efficient and less practical implementation. Thus, only when handling last-level cache (LLC) misses do we need to perform such translations. The tag controller therefore interfaces with the LLC and DRAM.

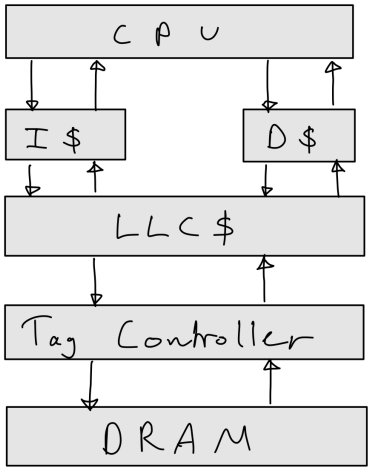

Figure 3: How the parts of the memory system interface.

2.1.2 CHERI

My project focuses on the CHERI architecture model. The model defines a single-bit tagged memory architecture, where 64-bit pointers can be replaced with 128-bit 'capabilities' that are also are given a 1-bit tag, which allows hardware-enforced security invariants to be guaranteed about these special values, and thus also about executions of programs that use them. There is more information about the CHERI model on the website for the Departement of Computer Science and Technology at the University of Cambridge [6].

Figure 4: A CHERI capability.

My project focuses particularly on the CHERI architecture as it is a speciality of the university and of my supervisor, and it is also the tagged memory architecture that I am most familiar with, as it was briefly introduced to us in IB, and I also spent some time working with it over the previous summer during my internship as a member of Arm's Morello Kernel Team. Much of my work and findings however will be transferable to other tagged memory architectures.

2.1.3 A Basic Scheme

The most basic scheme has a single flat table at the top of DRAM that contains all of the tags, and nothing more. This table contains one tag bit per 128 bits of regular memory (as a CHERI capability is 128-bits plus a 1-bit tag). The table therefore consumes the top 129^{th} of DRAM. The layout of DRAM is as depicted in figure 1.

When the tag controller must read from the table, it uses the following algorithm:

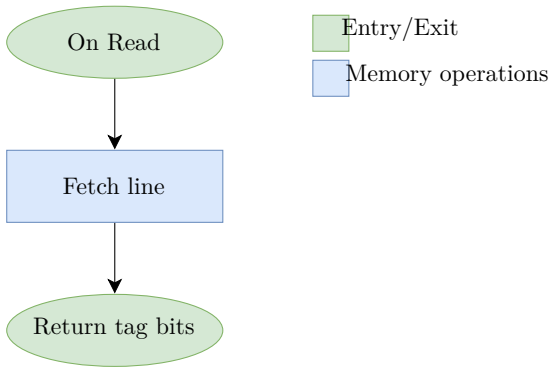

Figure 5: The basic algorithm for reading tags. Memory access operations are blue.

The algorithm for writing to the table is as follows:

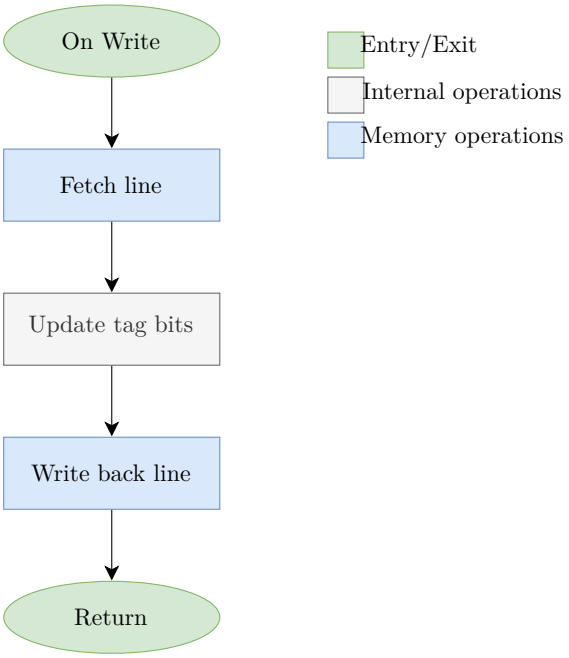

Figure 6: The basic algorithm for writing tags. Operations internal to the tag controller are grey.

As far as the processor and the cache systems are concerned, regular memory still begins at physical address `0x0`. The fact that the controller is mandating that the base of the *tag table* is at `0x0`, rather than the base of physical memory being here, is abstracted away by the controller's interface. For clarity, I will refer to the addresses that the processor and caches see as 'logical physical addresses', and those that the tag controller emits and DRAM sees as 'actual physical addresses'. These are both distinct from virtual addresses, which are not relevant here.
Also, note that everything here is done in terms of cachelines, as the tag controller's client is a cache – the tag controller receives logical base addresses of cachelines from the LLC, rather than those of individual values, and it must return entire cachelines to the LLC.

So, suppose the last-level cache misses the cacheline with logical base address a. It sends a request to the tag controller to retrieve the cacheline that lives at logical address a, and also its corresponding tags. The controller then uses the following formulae to calculate which actual physical addresses it needs to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{128} \tag{1}$$

$$\text{Actual Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} \tag{2}$$

Where the notation $\lfloor x \rfloor_y$ denotes the nearest multiple of y that is less than or equal to x – i.e. rounding x down to the nearest multiple of y.

For the actual base address of the value cacheline, we must take into account the offset of the value portion of DRAM. The value portion begins immediately after the end of the tag table. The tag table has size $\frac{\text{Physical Memory Size}}{128}$, thus we add this value onto the logical address of the line, a, to get the actual physical address of the line.
The actual base address of the tag cacheline will index into the tag table. In this scheme, the tags are arranged

in the same order as the 128-bit values are themselves in DRAM. That is, the first bit of the tag table is the tag of the first 128-bit value in DRAM. The second bit is that of the second 128-bit value, and so on. As we assume each cacheline is the industry standard of 64 bytes, and that each tag corresponds to 128 bits (16 bytes), there are 4 tags per value cacheline. Thus, **one cacheline of tags from the tag table** holds the tags for **128 value cachelines** (see equation 3) By dividing a by 128 we get the location in the tag table that corresponds to the location of the value in the value portion of DRAM. We then must calculate the base address of the tag cacheline which this location lies in, which is done by rounding down to the nearest multiple of 64, as the tag cachelines are 64 bytes.

$$\frac{512 \text{ tags per tag cacheline}}{4 \text{ tags per value cacheline}} = 128 \text{ value cachelines covered by one tag cacheline} \tag{3}$$

2.1.4 The Importance of a Tag Cache

On retrieving a cacheline, the tag controller then needs to access memory twice; once to retrieve the value cacheline, and once more to retrieve the tag cacheline, from which the appropriate tags are extracted and merged with the values, before returning this to its LLC client. This double access is where the hefty time overhead lies – having to access DRAM twice for every LLC miss would of course roughly double the the controller's response time. Without adding another level of data cache, we cannot avoid accessing the value cacheline in DRAM. What we can do though is introduce a new cache at this level of the memory hierarchy, just below the LLC and alongside the tag controller, that stores data from *the DRAM tag table*. This is known as the tag cache.

Now, we first check to see if the tag cacheline we want is in the tag cache. This request can be performed in parallel to the DRAM lookup of the value cacheline. If the tag cache happens to hit, then we avoid the second DRAM access entirely, and eliminate the overhead. Naturally, this motivates efforts to maximise the hit rate of the tag cache.

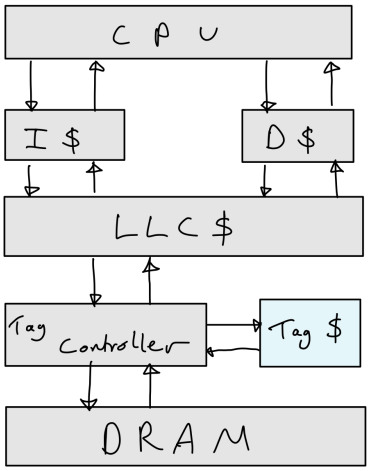

Figure 7: Where the tag cache sits in the memory system.

2.1.5 *Efficient Tagged Memory*'s Hierarchical Scheme

One of the tag controller schemes that my project focuses on is the scheme described in *Efficient Tagged Memory*. This is in part becuase of its simplicity, but also because my supervisor contributed to the paper; his familiarity

with the scheme allowed him to guide me easily and effectively where necessary.

One way to increase the hit rate of the tag cache is to store more information about the tag table within it. This can be done by simply increasing its size. However, there is an alternative – by adjusting the scheme, we can increase the 'cacheability' of the tag table. That is, we can fit more information about the tag table in the same size tag cache. Section VI of *Efficient Tagged Memory* details a tag table scheme that does just this.

This scheme uses a two-tiered hierarchical tag table. The 'leaf' table is identical to the table described in the basic scheme, with one tag bit per 128-bit value. The 'root' table contains bits that hold information about *the state of the tags in the leaf table.*
One bit in the root table corresponds to 512 bits in the leaf table – an entire cacheline of tags. The root bit is a 0 if all 512 bits in the corresponding leaf cacheline are 0, otherwise it is a 1.

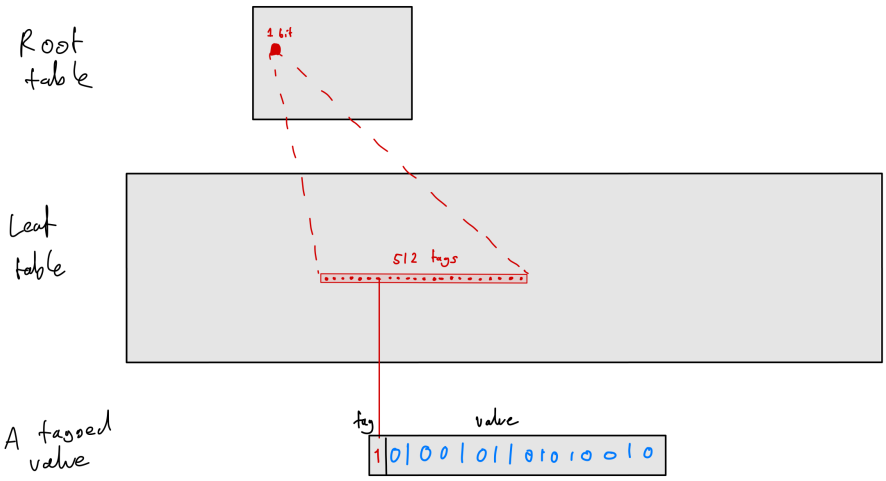

Figure 8: Efficient Tagged Memory's two-level hierarchical lookup table.

The tag controller has a new algorithm for both reading and writing the state of tags in the table. The algorithm for reading is as follows:

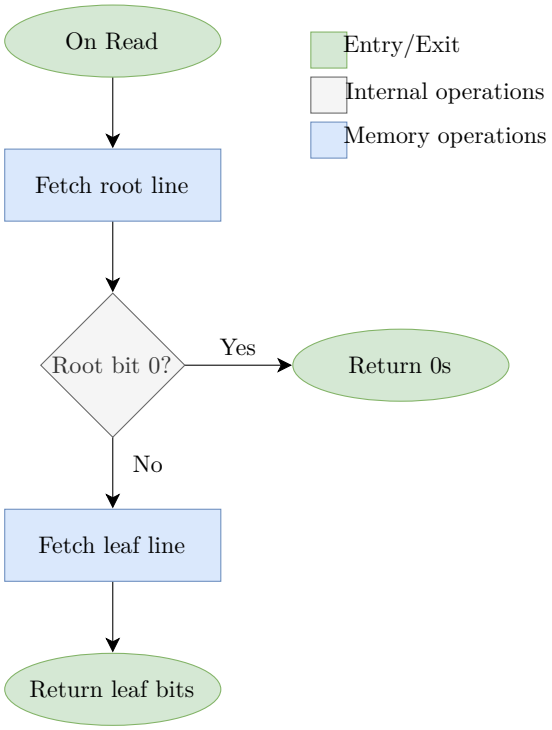

Figure 9: The ETM algorithm for reading tags.

The algorithm for writing is as follows:

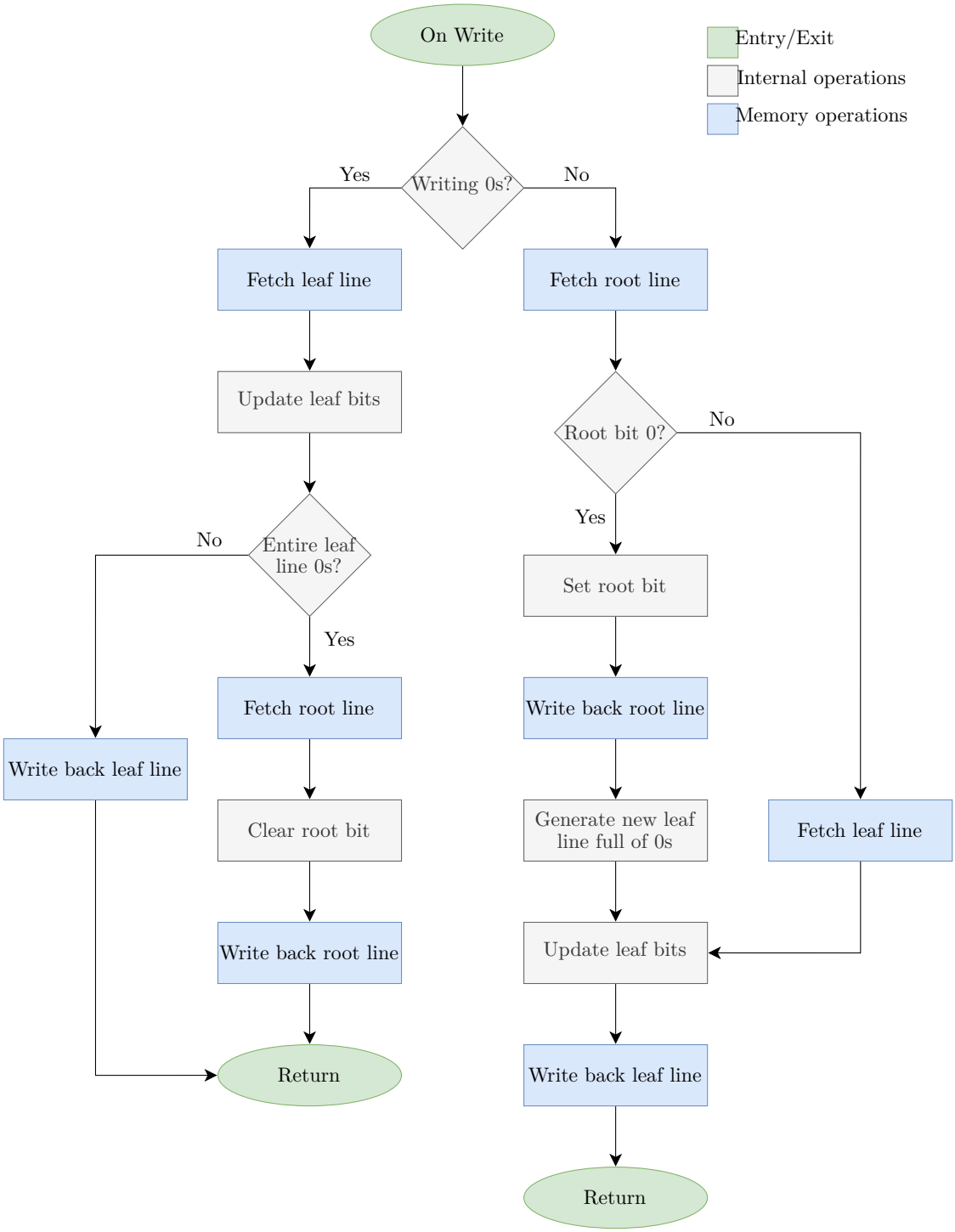

Figure 10: The ETM algorithm for writing tags.

The tag cache may hold lines from either the root table or the leaf table. For each *cleared bit from the root table* that resides in the tag cache, we are essentially containing information about the state of an entire leaf cacheline whilst only consuming one bit of cache space. When executing typical CHERI workloads, it is common to find cleared root bits. It is even common to find entire cachelines of cleared root bits. This is due to the nature of these workloads – the capabilities tend to be focused into small regions of memory, rather than spread out across the entire address space.
Thus, whilst in some cases we triple the number of DRAM accesses for a given LLC miss, the paper finds that there is only a very small overall increase in DRAM traffic overhead, due to the tag cache being able to hold plenty of information about the tags, and it therefore being scarcely necessary to perform additional DRAM accesses. Part of my project is to reproduce these results.

In DRAM, the root table lives above the leaf table. The leaf table lives above the value portion of DRAM (see figure 11).

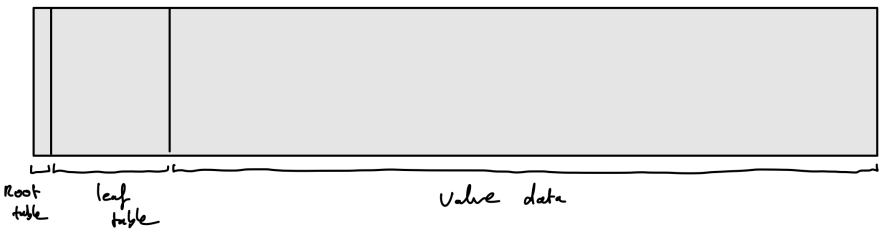

Figure 11: The physical layout of the hierarchical table.

Suppose again that the last-level cache misses the cacheline with logical base address a. The controller uses the following formulae to calculate which actual physical addresses it might need to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{65536} + \frac{\text{Physical Memory Size}}{128} \tag{4}$$

$$\text{Actual Leaf Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} + \frac{\text{Physical Memory Size}}{65536} \tag{5}$$

$$\text{Actual Root Tag Cacheline Address} = \left\lfloor \frac{a}{65536} \right\rfloor_{64} \tag{6}$$

The formulae for the actual base addresses of the value cacheline and the leaf tag cacheline are as they were in the simple scheme, with an additional constant of $\frac{\text{Physical Memory Size}}{65536}$ added on. This is to adjust for the offset that the root table adds.
The actual base address of the root tag cacheline will index into the root table. The address is calculated by similar logic to that of the leaf tag, and equation 7 explains the presence of the constant 65536.

$$\begin{aligned}\frac{512 \text{ leaf tags per leaf cacheline}}{4 \text{ tags per value cacheline}} &\times 1 \text{ leaf cacheline per root tag} \times 512 \text{ root tags per root cacheline} \\ &= 65536 \text{ value cachelines covered by one root cacheline}\end{aligned} \tag{7}$$

2.1.6 The Morello Project Scheme

As I mentioned prior, during my internship with Arm over the previous summer I contributed to their Morello project. Morello is a research project that defines a microarchitecture based on the CHERI model. Another tag

controller scheme that my project focuses on is the scheme used in the Morello microarchitecture. This is due to my own and my supervisor's familiarity with CHERI, and the Morello architecture and microarchitecture. There is more information about the Morello project at the project's website [7].

This scheme aims to do a similar thing to the one described in *Efficient Tagged Memory*, but in a different way. Instead of introducing a new, more information-dense table that can be stored in the existing tag cache, it introduces a new tag cache that holds information-dense data.

This scheme uses a simple flat DRAM tag table, just like the one in the basic scheme, but it employs two tag caches. One is a cache reserved for holding tag cachelines that contain *at least one* set tag, call this the non-zero cache. The other is a cache reserved for holding tag cachelines that contain *no* set tags – i.e. lines that are fully cleared, call this the zero cache. The former cache is just like a regular cache; for each cacheline, there is an entry in the cache that stores both the data at the address and the *address-tag** of the cacheline's base address. However, the latter cache only needs to store the address-tag of the cacheline's base address, as it must be that the entire cacheline contains zeroes, given that it is in the zero cache, thus there is no need to store any actual data. Requests can be issued to and handled by the two caches in parallel, further mitigating overhead.

* The *tag of a cacheline's base address* is defined as the upper K bits of the base address of the cacheline (for some K defined by various system parameters). This is the academic-standard name for this value, but it unfortunately clashes with the tagged memory *tags*. To disambiguate, I use the term *address-tag* hereon to refer to the former, and simply *tag* for the latter.

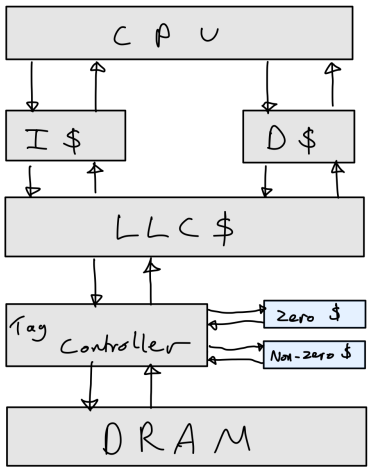

Figure 12: The layout of the two tag caches in the Morello memory system.

As the tag table has not changed from the simple scheme, the formulae used by the controller for translating addresses are the same (equations 1 and 2), and so are the two algorithms (figures 5 and 6).

2.2 My Project – A Tag Controller Simulator

The objective of my project was to develop a tag controller simulator in C++. When given a workload it should simulate the memory accesses that a specified tag controller algorithm would result in. The idea is that this simulator can be leveraged to provide insight into the performance and inner workings of the tag controller algorithm being simulated, providing useful information to an architect.

2.2.1 The Workloads

A former student had previously created a simulator that consumes a program binary and simulates the execution of the binary on a CHERI architecture. The simulator outputs trace files that log the last-level cache misses that occurred during the simulated execution.

I have been given access to a collection of trace files by my supervisor. These files were logged whilst running various benchmark binaries. The simulator I developed takes these trace files as input. As the tag controller only receives information about LLC misses, the trace files contain sufficient information to determine which accesses would be made by the simulated tag controller algorithm if it were to run the given workload.

2.2.2 ChampSim – An Appropriate Cache Simulator

ChampSim is an open-source trace based cache simulator written in C++. There is more information about ChampSim at the ChampSim GitHub profile page [8].

My simulator interfaces with ChampSim, having it handle the simulation of the tag cache(s). My simulator provides ChampSim with a trace file of memory accesses that the simulated tag controller makes during its execution of the workload. ChampSim then provides the user with metrics regarding cache access frequency and hit ratio. The user can specify parameters of the tag cache such as cache size and number of ways.
hampSim is a good choice of cache simulator because its open-source C++ code allowed me to easily extend it to add necessary instrumentation. Also, it has built-in cache prefetchers which can be enabled or disabled simply by adjusting a configuration file. This lets an architect see how much difference adding a prefetcher to their tag cache would make to performance.

I opted to use an existing cache simulator instead of developing my own because the focus of this project is on investigating tag controllers, as opposed to caches in general, and my supervisor informed me that there are likely many opportunities for bugs to arise when developing a cache simulator of my own, and thus it is a more appropriate use of my time to focus more on tag controller research rather than debugging software that has already been written.

2.2.3 Visualisation Tool

It would be useful for an architect to be able to somehow visualise how the state of the tag table and the tag cache changes as the workload executes. Having this insight would inform new approaches and justify design choices for a novel tag cache.

As part of my project, I developed a tool that allows a user to take ‘snapshots’ of the tag cache at chosen points of workload execution, and view them afterward. The images depict the state of the tag table and the tag cache at that point of execution.

2.2.4 Requirements Analysis

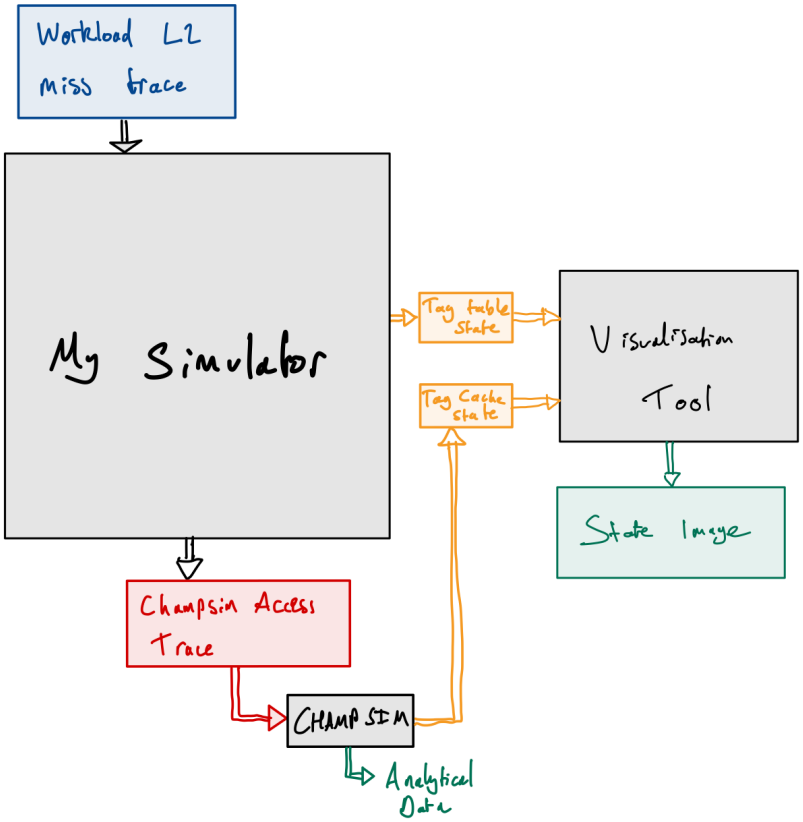

Figure 13: A black-box diagram explaining the inputs and outputs of the pieces of the tag controller evaluation framework.

There are several requirements that a successful simulator must meet:

- It must be able to decode the LLC miss input trace file format.
- It must be able to correctly simulate the sequence of memory accesses that the simulated tag controller algorithm would perform when executing the given workload.
- It must be able to produce an accurate output trace file of these accesses that is decodable by ChampSim.

On top of this, for the visualisation tool to be functional and useful, the following are required:

- My simulator must be able to output the state of all tags at given points of execution.
- The system must be able to extract the state of the tag cache from ChampSim at given points of execution.
- The visualisation tool must be able to read in both types of captured state and produce an image that accurately depicts it all.

2.3 Starting Pont

The following bullet points detail relevant work completed and preparations made before beginning the project:

- I had read the paper *Efficient Tagged Memory*.
- As I have mentioned, over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm's Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model.
- My project uses ChampSim, the open-source cache simulator. I also extend this program by adding logging support, by branching from commit #2b8d3fc28abb6072d7675228418fa7cfe862d4dc. I took no part in contributing to any versions of ChampSim prior to this commit.

3 Implementation

This chapter !!!.

3.1 Implementation Overview

The simulator consists of 3 main parts: the trace decoder, the tag controller algorithm, and the logical tag cache. Figure **??** shows how these parts fit into the information flow of the whole project.

[FIGURE WBX!!!]

3.2 Simulator Framework

The simulator is a program with a command-line interface defined in `main.cpp`. It allows a user to specify the following:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file
- the path of the output trace file
- the maximum number of LLC misses to process (optional, default 5 million)
- the path of the output log file to log the tag table state into (optional)
- the indices of the instructions after which a tag table state log entry should be made (known as 'log points'; optional)

`main.cpp` then parses these arguments, and opens the files specified. It creates an instance of a `Decoder` and has it decode the input files. It then creates an instance of a `Controller` of the specified scheme, and a `Simulator`. It then has the `Simulator` simulate the tag controller's execution when running under the workload of the input trace files.

The `Simulator`'s job is simply to iterate through the decoded LLC misses, passing each to the controller's `handleMemoryAccess` method. This simulates the LLC's stream of LLC misses being passed to the tag controller in a hardware implementation.

[CODE SIM!!!] the simulator's only method

At the end, `main.cpp` reports the total number of LLC misses that were handled, and the total number of memory accesses the tag controller made. It also saves and closes the output file(s).

3.3 Trace Decoder

The `Decoder`'s job is to retrieve information from the input trace files and provide it to whichever calling function requested it.

On its construction, it is given the file objects for the two input trace files. It has two public methods, `readLLCMisses` and `getTags`.

`readLLCMisses` is what `main.cpp` uses during setup to decode the sequence of LLC misses from the LLC miss input trace file. The file contains a sequence of structs with no padding between them. The structs have fields which begin at specific byte indexes, as detailed in figure **??**.

[FIGURE LFM!!!] The format of a struct in the LLC miss file (see obsidian log)

By using the correct format, the decoder reads out the fields of each LLC miss entry from the file, one at a time. For each entry, it constructs an `llcMiss` struct, which is defined in `trace.h`.

The final step of this process is to *furnish* the decoded `llcMiss` entry and convert it to a `memAccess` struct (also defined in `trace.h`) before writing that into the buffer. *Furnishing* refers to a process of filling in partially-unknown information about the tags as best we can.
To explain this, we must first understand some background about the source of the input trace files. The CHERI simulator that generated the input trace files worked like so: Instructions were decoded from the program binary, and issued to the simulated CPU. In the case of memory access instructions that resulted in LLC misses, an entry was made to the LLC miss trace file, detailing the miss. The entry attempts to describe the state of the four tag bits that were read (in the case of a read), or the state that we update the four tag bits to have (in the case of a write). However, the information about the state of *all* four tag bits is not necessarily known at the time the entry was logged. At this time, the simulator only had information about the tag that was read/written by the instruction that caused the miss, and information about the other tags that could be inferred from previous memory access instructions – for example, suppose that a previous instruction had set tag 2 of a cacheline to 1. Suppose a later instruction reads tag 3 of this same cacheline; we know the value of tag 3 from the instruction's result, and we also know that tag 2 has value 1, assuming no other instruction has caused it to change since the previous instruction set it. In the case of the other two tags in the cacheline, the simulator does not know what value they have at this time.
This lack of knowledge about particular tags is embedded in each entry in the LLC miss trace file, and upon being decoded, it is stored into the `tags_known` field of the resulting `llcMiss` struct. This field is a bitmask indicating for which of the four tags we have definite information, and for which we do not.
To attempt to gain information for tags that we do not know about, we can refer to the initial state file that is associated with the LLC miss trace. This file contains information regarding what can be known about the *initial state of the tags* in memory. In this file, there is an entry for each tag in the DRAM tag table. The `Decoder`'s private method `getInitialTag` is responsible for approximating a tag of interest based on the initial state file's entry for that tag.
The `getInitialTag` method first reads and decodes the entry for the tag of interest, obtaining two fields: a boolean named `tag` and a value of the enumerated type `initialAccessType`, which is defined in `trace.h`. The following table details how the `assumeTag` function generates a best-guess initial tag value from these two fields:

Initial access type	Tag	Explanation	Assumed tag
INITIAL_ACCESS_TYPE_CLOAD	0	At some point, before the tag was modified, a capability-load instruction read this tag's state as a 0, so the tag was definitely initially 0.	0
INITIAL_ACCESS_TYPE_CLOAD	1	At some point, before the tag was modified, a capability-load instruction read this tag's state as a 1, so the tag was definitely initially 1.	1
INITIAL_ACCESS_TYPE_CSTORE	0	A capability-store instruction overwrote this tag's state to 0 before the initial value was discovered. We assume the capability-store did not change the value.	0
INITIAL_ACCESS_TYPE_CSTORE	1	A capability-store instruction overwrote this tag's state to 1 before the initial value was discovered. We assume the capability-store did not change the value.	1
INITIAL_ACCESS_TYPE_LOAD	0/1	No capability instructions were performed on this tag, so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0
INITIAL_ACCESS_TYPE_STORE	0/1	No capability instructions were performed on this tag, so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0

Putting it all together, the `furnish` method converts decoded `llcMiss` structs into `memAccess` structs by using the `getInitialTag` method to fill in gaps. The `getInitialTag` method in turn uses the assumption scheme defined by `assumeTag` to make an educated guess at what the tag previously was, though at times it must make assumptions due to lack of information.

The `Decoder` class also has a `getInitialTags` method that produces an entire cacheline's worth of initial values of tags in the tag table. It uses the same methods, and the same assumptions for making educated guesses, as the `getInitialTag` method does. It is used by `Controllers` when they initialise the state of their `Cache`. This method is more efficient than simply calling `getInitialTag` 512 times as it performs a single disk read to retrieve all 512 relevant entries from the initial state file at once. It also provides a more useful interface to the calling methods, as they operate on cacheline-granularity rather than bit-granularity.

3.4 Tag Cache

`Cache.h` defines the `Cache` class, which represents a *logical* cache. By logical, I mean that it does not simulate the operations of a cache such as cacheline eviction and replacement – that is ChampSim's job – but instead it provides an interface to the tag controller that mimics the interface of a real cache, and performs other duties important to the simulation.

This class is responsible for the following:

- Provide an interface to the tag controller that mimics a real cache
- Log all memory accesses that the tag controller makes

- Keep track of the state of the DRAM tag table as the simulation progresses

The `Cache` provides a `doRead` and `doWrite` method to the tag controller, allowing it to read from memory. From the point of view of the controller, the logical cache always hits. There is no need to determine whether or not the cache would hit inside this simulation – the purpose of this simulation is to determine the *sequence of **memory addresses** that the tag controller attempts to access* when running under the chosen workload. This is independent of whether or not the cache hits or misses, thus the control flow will always be correct, and therefore the output trace produced will still be an accurate trace of the sequence of memory address accessed by a real controller.
After we produce the access trace, we can feed it into ChampSim. ChampSim then accurately simulates the physical cache, simulating which lines would hit, which would miss, which would be evicted, and so on, all the while recording statistics such as access count and hit count.
Figure **??** details how information regarding memory accesses flows through the simulations, and which parts are responsible for simulating which functionality of the real system.

[FIGURE CCH!!!] read the bit that ref's it

Each time the tag controller needs to read a tag cacheline from memory, we want to create an entry into the output trace reflecting this. As well as allowing the tag controller to read from and write to the state of the tag table, the `doRead` and `doWrite` methods also take the responsibility of logging the addresses accessed to the output log. They do this by calling the private `logRead` and `logWrite` methods, which simply append a populated `champsimInstr` struct to the file, building up a trace file that ChampSim can read.

There are also cases where we want to read from or update the state of the tag table without writing an entry to the output trace file. I mention these cases later. The `peek` and `set` methods allow a caller to read from and write to the tag table state respectively.

The `Cache` class has two getter methods – `getNumAccesses` and `getPrevLogged`. The former returns the number of entries made into the output trace file. The latter returns a vector of all entries that have been made – this is used when testing.
The class also has a method `dump`, which writes each and every bit of the current state of the tag table into the tag table state output log file.

3.5 Tag Controller

[CODE CTR!!!] controller.h class

A tag controller is implemented as a class that derives from the abstract `Controller` class. Each tag controller has a (logical) `Cache`, which it sends requests to during its operation. This logical 'has-a' relationship exactly matches the memory system model described in the preparation section (see figure 7).

Being abstract, this class also details the methods a concrete tag controller must implement – it must have a `setupCache` method, and `handleMemoryAccess` method.
The `setupCache` method is to be called immediately after the `Controller` instance is constructed, so that the state of its cache is correctly initialised, with help from the passed `Decoder` that reads from the initial tag table state input file. This method is abstract because different tag controllers are part of different schemes, and different schemes have different tag table layouts (the Morello Project's flat table layout from the *Efficient Tagged Memory*'s hierarchical table layout, for instance), thus the algorithm to initialise them will differ.
The `handleMemoryAccess` method implements the algorithms that the tag controller follows when handling LLC misses. As this simulator is only concerned with tags and not actual value data, a concrete implementation of this method only needs to perform the tag access part of the algorithm, and not the value access or the merging of values with tags.

The `Controller` class also defined three methods that allow external callers to interact with the state of the tag cache. These methods are simply proxies to public methods of its `Cache`.

3.5.1 Flat Table Controllers

`FlatTableController.h` defines an abstract class `FlatTableController` that extends `Controller`. This class partially defines a specific type of tag controller – those that use a flat table, such as the controllers from the basic scheme and the Morello Project scheme. Controllers of this type have two things in common: how logical addresses are translated into actual ones, and how the tag cache should be initialised at the beginning of a simulation.
The translation from a logical cacheline base address to the corresponding actual tag cacheline base address is detailed in equation 2. `FlatTableController`'s `translateToTagAddr` method does exactly this (see figure **??**). For a flat table controller, the tag cache is initialised by simply iterating across the entire initial tag table state and populating the cache's cachelines.

[code FLT!!!] code for translateToTagAddr

By implementing a single abstract class from which several other classes can inherit instead of duplicating the functionality across several base classes, I reduce the size of the codebase and increase maintainability due to only having to change a single implementation when changes are necessary. It also becomes true that by testing this functionality for a single base class, I test it for all of them simultaneously.

3.5.2 Basic Scheme

`BaselineController.h` defines a concrete class `BaselineController` that implements the tag controller from the basic scheme. The class inherits from `FlatTableController`.

The implementation of `handleMemoryAccess`, the final remaining abstract method to be implemented, is very simple, and it is clear how the code directly corresponds to the algorithms from figures `fig:bar` and `fig:baw`.

[code BAS!!!] code for baseline controller

3.5.3 *Efficient Tagged Memory*'s Hierarchical Scheme

`ETMController.h` defines a concrete class `ETMController` that implements the tag controller from the scheme described in `Efficient Tagged Memory`.

This class's `setupCache` method instantiates the leaf table in the same way that a flat table controller instantiates its table. Whilst it does this however, it also populates its root table. Each time a cacheline of tags is placed into the leaf table, we keep track of whether or not the cacheline contained any set tags, recording a 1 in the current byte if there were any, and a 0 if not. After processing 8 leaf table cachelines, we push the current byte into the current cacheline. After pushing 64 bytes, we set the now-completed cacheline in the root table.

[code ETS!!!] etm's setupCache

This class defines two auxiliary functions, `translateToLeafAddr`, and `translateToRootAddr`. Just as `FlatTableController` `translateToTagAddr` implemented the appropriate equation from the Preparation chapter to translate an logical cacheline base address to the corresponding actual tag cacheline base address, `translateToLeafAddr` implements equation 5 to translate a logical cacheline base address to the corresponding actual tag cacheline base address in the *leaf table*. Similarly, `translateToRootAddr` implements equation 6 to translate that cacheline to the corresponding actual cacheline in the root table instead – that is the base address of the line in the root table that contains the bit whose scope covers the corresponding leaf table tags.

[code ETT!!!] the two etm aux methods

The implementation of `handleMemoryAccess` is more complex than that of the basic tag controller, but once again the code directly corresponds to the algorithms from the appropriate figures (`fig:etr` and `fig:etw`).

[probably put the implementation in an appendix, and refer to it!!!]

3.5.4 The Morello Project Scheme

`MorelloController.h` defines a concrete class `MorelloController` that implements the tag controller from the Morello Project scheme. The class inherits from `FlatTableController`.

The Morello project tag controller implementation is a special case. The Morello Project's microarchitecture uses two distinct caches. My program handles this by producing the access logs for the two caches in two separate simulations. On invoking the program, the user specifies whether they want to simulate the zero cache, or the non-zero cache. The simulator will then run and produce an output trace file that logs the accesses made only to the specified cache. For example, if the user specifies that they want to simulate the non-zero cache, then the output trace produced will detail all accesses made to *non-zero* tag cachelines, and not any to *zero* cachelines.
This way, the simulation framework does not need to be adapted to handle multiple caches at once, and the output log can still be passed straight to ChampSim to simulate the chosen cache.

Now though, this means that the algorithms that we impelment must change. In simulation, we want the tag controller to only make a memory access if the line we are accessing is a line held in the cache we are simulating. The `Cache`'s `doRead` method allows us to see the contents of a tag cacheline, but it also creates an memory access entry in the output trace file. This is a problem if the line happens to be one that we do not want to log the access of – with this method, we can only know if we want to log the access after we have called the method and consequently logged the access.
The solution is to instead use the `Cache`'s `peek` method. With this method we can see the state of the cacheline without logging the access, and then only after we decide if we want to log the access do we actually log it.
Similarly, on updating a tag, we might not necessarily want to log the write memory access. But it is important that we update the state of the tag in the simulation, otherwise the algorithm might later read stale data and make decisions that are not accurate to a real world run of the controller, where the value did in fact change in memory. The `Cache`'s `set` method can be used to update the state of a cacheline without logging a write memory access to the output trace.
Figures 14 and 15 detail the new algorithms we must implement. They contain 'pseudo-memory operations' – these are purely for simulation control flow purposes and do not correspond directly to real accesses that a physical tag controller would make.

The algorithm for reading is as follows:

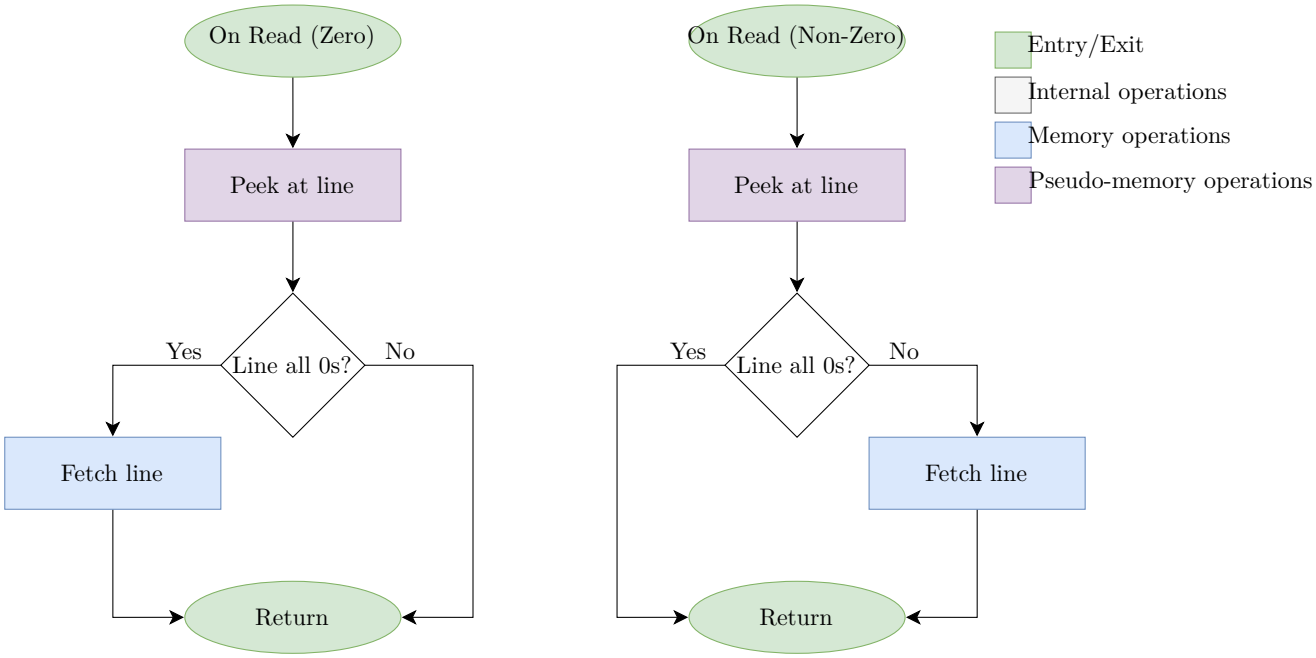

Figure 14: The Morello Project algorithm for reading tags. Pseudo-memory access operations are purple.

The algorithm for writing is as follows:

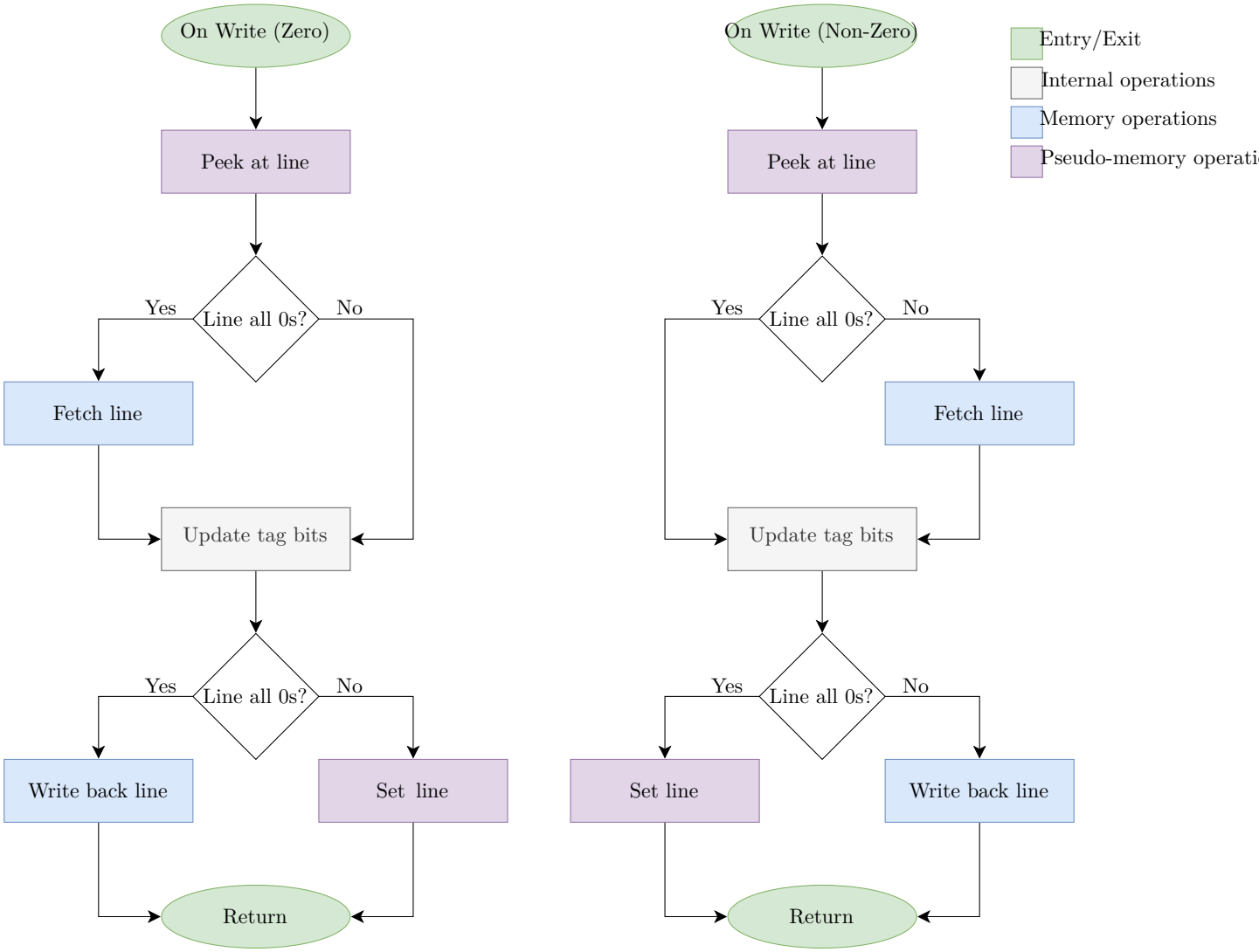

Figure 15: The Morello Project algorithm for writing tags.

The implementation of `handleMemoryAccess` implements these algorithms. Lines !!! show that after having `peek`ed at the state of the cacheline, we only actually log the memory access if the state of the cacheline matches with the cache that we are simulating. Lines !!! show that depending on whether or not we want to record the memory access or simply update the simulation's state, we call either `doRead` or `set`.

3.6 Logging

3.6.1 Logging Tag Table State

Another job of the `Simulator` class is to stimulate the `Cache`, via the `Controller`'s proxy method, to dump its tag table state once a log point is reached.

When it does this, it reports the index of the most recent memory access request. These indices are used later by the instrumented version of ChampSim. We also want ChampSim to log the state of the tag cache at this same point of program execution. Therefore by keeping track of the index of the memory access immediately prior to the log point, when ChampSim runs its simulation of the cache, it can know that once it reaches the memory access with an equal index, it must dump the current state of the cache.

3.6.2 Adding Logging to ChampSim for Tag Cache State

To add cache state logging support to ChampSim, I had to modify three files.

In `cache.h` I simply made the `BLOCK` struct public so it could be accessed from outside of the class.
In `main.cc` I added to option to specify two extra command line arguments:

- the path of the output log file to log the tag cache state into
- the indices of the memory accesses after which a tag cache state log entry should be made

The `main` function passes this information to the `champsim::main` function defined in `champsim.cc`.
This function was then modified to check after each phase of processing if it has reached a log point. If it has, it dumps all of the addresses in the cache into the output log file.

3.7 Visualisation Tool

With the ability to grab the state of the tag table and tag cache at any point in the execution of a workload, we can now create a tool to visualise the this state, effectively showing us a 'snapshot' of the tag state at that moment in execution.

`visualiser.py` combines the simulation and the visualisation into one package. It allows the user to specify the following command line arguments:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file
- the maximum number of LLC misses to process
- the indices of the instructions after which a snapshot should be made
- the string prefix of the output image filenames

[FIGURE VIS!!!] the white-box implementation of the visualiser (remember that visualiser and everything else is inside the overarching box called visualisation tool)

The program first invokes the tag controller simulator, passing it the arguments it requires. The simulation finishes, and the output files are produced. The program then reads the standard output of the tag controller simulation, reading the reported memory accesses after which ChampSim must dump the state of the tag cache. Next, it invokes the instrumented version of ChampSim, passing in the required arguments. ChampSim simulates the operation of the tag cache, and logs the state of the cache at the desired points. It then finishes, reporting its usual metrics to the user.
The program now has the logged states of both the tag table and the tag cache. It finally generates a PNG showing visually this state. One PNG is generated for each log point.

3.8 Testing

3.9 Repository Overview

- the dirtree thing

4 Evaluation

5 Conclusions

References

[1] Alexandre Joannou et al. “Efficient tagged memory”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.

[2] Brian E. Clark and Michael J. Corrigan. “Application System/400 performance characteristics”. In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.

[3] Jedidiah R Crandall and Frederic T Chong. “Minos: Control data attack prevention orthogonal to memory model”. In: *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE. 2004, pp. 221–232.

[4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.

[5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.

[6] *https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/*.

[7] *https://www.morello-project.org/*.

[8] *https://github.com/ChampSim/*.

Appendices

Index (Optional)

Project Proposal